

Stored Procedures en SQL Server

Con esta clase entramos a un módulo nuevo y a un concepto que es, sin exagerar, la base de todo lo que viene de aquí en adelante. Hasta ahora hemos escrito consultas, creado vistas, definido triggers. Todo eso está bien, pero cada uno resuelve un pedazo del problema. Un procedimiento almacenado — o SP, como lo vamos a llamar de ahora en adelante — puede coordinar todos esos pedazos en un solo lugar.

Pensemos en algo concreto. El proceso de cierre de mes de TiendaLatam probablemente involucra varias operaciones: calcular ventas, actualizar estados, generar resúmenes. Hoy eso implica abrir quince scripts, ejecutarlos en orden y esperar que nadie se equivoque de archivo ni se salte un paso. ¿Qué pasaría si todo eso se ejecutara con un solo comando? Algo como `EXEC SP_CierreDeMes '2025-01-01', '2025-01-31', 'AR'`. Un comando, un resultado, con toda la lógica centralizada y probada. Eso es un procedimiento almacenado.

Qué es un SP y por qué es distinto a una consulta suelta

Un SP es un bloque de código SQL con nombre que se guarda en la base de datos. Se ejecuta con el comando `EXEC`, puede recibir parámetros de entrada y devuelve resultados. Hasta ahí podría sonar parecido a una vista, pero las diferencias son importantes.

La primera es el rendimiento. Cuando ejecutas una consulta suelta, SQL Server tiene que analizarla, compilarla y generar un plan de ejecución cada vez. Con un SP, ese plan se genera la primera vez y queda guardado en caché. A partir de ahí, las ejecuciones siguientes son más rápidas porque SQL Server ya sabe cómo resolver esa consulta.

La segunda es el mantenimiento. Si tienes una consulta de ventas por país copiada en diez scripts diferentes y cambia la lógica, tienes que actualizar los diez. Con un SP, la lógica vive en un solo lugar. La actualizas una vez y todos los que lo ejecutan obtienen la versión nueva.

La tercera es el control de acceso. Puedes darle a alguien permiso de ejecutar el SP sin darle acceso directo a las tablas. Esto es parecido a lo que vimos con las vistas, pero los SP son más flexibles porque pueden recibir parámetros, hacer validaciones y coordinar múltiples operaciones.

Y la cuarta es la reutilización. Un SP lo puedes llamar desde SQL Server Management Studio, desde una aplicación, desde Power BI, desde una API, o incluso desde otro SP. Es una pieza que se conecta con todo.

Crear un SP paso a paso

La estructura de un SP tiene partes claras. Primero viene la declaración: `CREATE PROCEDURE`, seguido del nombre. La convención es usar el prefijo `SP_` para identificarlos fácilmente, igual que usamos `VW_` para las vistas y `TR_` para los triggers.

Después del nombre vienen los parámetros. Son las variables que el SP recibe cuando lo ejecutas. Cada parámetro tiene un nombre (que empieza con `@`), un tipo de dato, y opcionalmente un valor por defecto. En la clase creamos un SP llamado `SP_ConsultarVentasPorPais` con tres parámetros: `@CodigoPais`, `@FechaInicio` y `@FechaFin`.

Después viene la palabra `AS`, luego `BEGIN`, y dentro va toda la lógica que quieres que el SP ejecute. En nuestro caso, es un `SELECT` con `JOINS`, filtros y agrupaciones — la misma consulta que un analista tendría que escribir a mano cada vez. La diferencia es que ahora esa lógica vive dentro del SP, y para obtener el resultado solo hay que escribir `EXEC SP_ConsultarVentasPorPais 'AR', '2024-01-01', '2024-12-31'`.

Un detalle que vas a ver en todos los SP y que conviene entender desde ahora: la línea `SET NOCOUNT ON` al inicio del bloque `BEGIN`. Lo que hace es decirle a SQL Server que no envíe los mensajes de "N filas afectadas" después de cada operación. En un SP simple con un solo `SELECT` quizás no importa mucho, pero cuando el SP tiene cinco operaciones internas, cada una genera un mensaje adicional que viaja por la red. Si hay una aplicación ejecutando ese SP en un loop, esos mensajes se acumulan y afectan el rendimiento. La convención es ponerlo siempre — es una línea y no tiene costo.

Ejecutar un SP y pasarle parámetros

Una vez creado, el SP aparece en el explorador de SQL Server Management Studio, dentro de la carpeta "Programación > Procedimientos almacenados". Para ejecutarlo usas **EXEC** seguido del nombre y los valores de los parámetros.

Puedes pasarle los parámetros de dos formas. La primera es por nombre: **EXEC SP_ConsultarVentasPorPais @CodigoPais = 'AR', @FechaInicio = '2024-01-01', @FechaFin = '2024-12-31'**. La segunda es por posición: **EXEC SP_ConsultarVentasPorPais 'CL', '2024-01-01', '2024-12-31'**, donde los valores se asignan en el mismo orden en que se declararon los parámetros.

En la clase lo probamos con Argentina, después con Chile, después con Colombia — cada vez cambiando solo los parámetros. Eso es la gracia: la lógica es siempre la misma, lo que cambia es la entrada. Si mañana necesitas conectar esto con Power BI, simplemente le pasas el SP y los parámetros, y el resultado llega sin que nadie tenga que escribir la consulta completa.

Valores por defecto: hacer el SP más flexible

Hay situaciones donde no siempre vas a querer pasar todos los parámetros. Quizás la mayoría de las veces solo necesitas consultar por país y que el rango de fechas sea automáticamente el último año. Para eso existen los valores por defecto.

Cuando declaras un parámetro, puedes asignarle un valor que se usa si nadie le pasa nada. En la clase hicimos que **@FechaFin** tome por defecto la fecha actual y que **@FechaInicio** tome un año atrás. Así, si ejecutas el SP pasando solo el código de país, automáticamente te trae el último año. Y si no le pasas ni siquiera el país, te trae todos los países del último año.

Eso convierte un SP rígido en una herramienta flexible. El mismo SP sirve para la consulta rápida del analista que solo quiere ver México, para el reporte mensual que necesita un rango específico, y para el dashboard que quiere ver todo. No son tres SP distintos — es uno solo con parámetros inteligentes.

Mantenimiento: modificar, eliminar y explorar

Al igual que con las vistas, los SP se pueden modificar, eliminar y explorar.

Para modificar usas `ALTER PROCEDURE`, que reemplaza la lógica interna sin perder los permisos que ya tenías asignados sobre el SP. Existe también `CREATE OR ALTER PROCEDURE` (disponible desde SQL Server 2016), que es más práctico: si el SP no existe lo crea, y si ya existe lo modifica. Es el comando que vas a usar más en el día a día.

Para eliminar: `DROP PROCEDURE IF EXISTS` seguido del nombre. Y si quieres ver qué query tiene adentro un SP que creó otra persona o que tú creaste hace meses y ya no recuerdas, `EXEC sp_helptext 'NombreDelSP'` te devuelve la definición completa.

Para dar permisos de ejecución a un usuario: `GRANT EXECUTE ON NombreDelSP TO NombreDelUsuario`. Los permisos los vamos a trabajar con más detalle en módulos posteriores, pero es bueno saber que la sintaxis es así de directa.

El SP como contrato

Hay una forma de pensar en los procedimientos almacenados que ayuda a usarlos bien: un SP es un **contrato**. Acepta ciertos parámetros, ejecuta una lógica definida y devuelve un resultado predecible. Quien lo ejecuta no necesita saber qué hay adentro — solo necesita saber qué parámetros pasarle y qué esperar de vuelta.

Eso es lo que los diferencia de una consulta suelta: la consulta es una instrucción abierta, el SP es una promesa cerrada. Y esa claridad es lo que permite que un SP sea la pieza central de automatización en cualquier base de datos de producción.

En la próxima clase vamos a hacer que los SP sean más inteligentes, agregándoles control de flujo con `IF`, `ELSE`, `WHILE` y variables internas. Pero antes de llegar ahí, necesitabas dominar la base: crear, parametrizar y ejecutar. Eso ya lo tienes.

Desafío de cierre

Crea el SP `SP_ConsultarVentasPorPais` con los parámetros que vimos en clase (código de país, fecha de inicio, fecha de fin, con valores por defecto). Después, extiéndelo para que además del resumen de ventas también devuelva el nombre del empleado con más ventas en ese país y período. Pruébalo con al menos tres combinaciones de parámetros distintas y comparte tus resultados en los comentarios de la clase.